

逻辑回归

逻辑回归可以看作是回归和分类之间的桥梁，是线性回归算法的一种拓展，当我们想将线性回归应用到分类问题中该怎么办呢？比如二分类问题，将 X 对应的 y 分为类别 0 和类别 1。

线性回归本身的输出是连续的，也就是说要将连续的值分为离散的 0 和 1。答案很容易想到，找到一个**预测分类函数**，将 X 对应的 y 映射到(0,1)之间。这就是逻辑回归的核心思想。

假设现在有一些数据点，用一条直线对这些点进行拟合，这个拟合过程就称作回归。而逻辑回归是一种预测分析，解释因变量与一个或多个自变量之间的关系。与线性回归不同的是它的目标变量有几种类别，所以逻辑回归主要用于解决分类问题。与线性回归相比，它使用概率的方式，预测出属于某一分类的概率值。如果概率值超过 50%，则属于类别 1，否则属于类别 0。

1 逻辑回归的基本原理

逻辑回归虽然带有回归的名称，但它是用来分类的模型。简单来说，逻辑回归是一种用来解决**二分类问题**的机器学习方法，用于估计某事件发生的可能性。比如某用户购买某商品的可能性。逻辑回归通过将样本的特征与样本发生的概率联系起来，输出样本可能发生的概率，根据输出概率的数值大小来推断新样本是属于哪一类别。如果输出的概率值大于 0.5，那么新输入的样本就属于这一类别，否则就不属于这一类别（逻辑回归用来判定类别的阈值不是只能设置 0.5，该阈值是一个超参数，可以自己调整）。

对于机器学习来说，本质就是求出一个函数 f ，有一个样本 x 输入到函数 f 中，最终得到的值通常是 $\hat{y}$ 。在逻辑回归中，得到的是一个概率值 $\hat{p} = f(x)$ ，即对于逻辑回归来说，是要得到一个函数 f ，把新样本 x 放入到函数 f 中， $f(x)$ 会计算出一个值 $\hat{p}$ ，我们会根据概率值 $\hat{p}$ 来对样本进行分类。以上过程描述可用表达式表示如下：

$$\hat{y} = f(x) = \begin{cases} 1, & \hat{p} \geq 0.5 \\ 0, & \hat{p} \leq 0.5 \end{cases}$$

逻辑回归既可以看作是一个回归算法，也可以看作是一个分类算法。如果不进行最后一步，根据 $\hat{p}$ 的值进行分类操作的话，此时就是线性回归算法，输出的结果是个连续的值；如果进行最后一步，此时就是逻辑回归算法，计算出的是通过样本的特征来拟合计算出一个事件发生的概率。比如给出一个用户的信息，计算用户是否购买某商品的概率。通常情况下，使用逻辑回归都是用来当作分类算法用。通过以上的例子也可以看出，分类只可以解决二分类问题，如果想要解决多分类问题，可以使用一些其他的技巧进行改造。

2 逻辑回归的计算步骤

按照逻辑回归的原理，求解过程可以分为以下三步：

(1)**找一个合适的预测分类函数**，用来预测输入数据的分类结果，一般表示为 f 函数，需要对数据有一定的了解和分析，然后确定函数的可能形式。

(2)**构造一个损失函数**，该函数表示预测输出值 $\hat{y}$ 与训练数据类别真实值 y 之间的偏差，一般是预测输出与实际类别的差，可对所有样本的偏差求 R^2 值等作为评价标准。

(3)**找到损失函数的最小值**，因为值越小表示预测函数越准确。求解损失函数的最小值是采用梯度下降法进行求解的。

2.1 预测分类函数

逻辑回归是用什么样的方式来得到一个事件发生的概率呢？

线性回归可表示为 $\hat{y} = f(x)$, $f(x)$ 也可表示成 $\hat{y} = X_b \cdot \theta$ 。

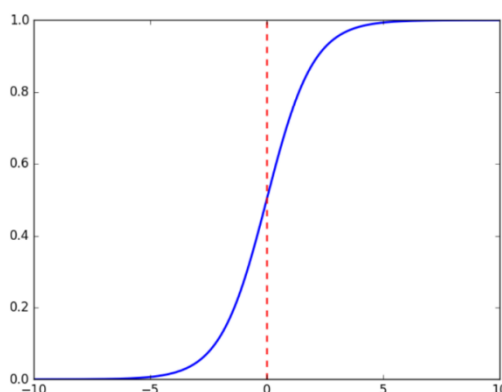
线性回归得到的值域是 $(-\infty, +\infty)$ ，即通过线性回归算法可得到一个任意的值。我们知道概率的值域是在 $[0,1]$ 之间取值，所以如果直接使用线性回归的方式，看能不能找到一组参数 θ ，特征 X_b 和这组 θ 点乘之后得到的值来表达这个事件所发生的概率。

解决方案也非常简单，使用线性回归的表达式，用 X_b 和 θ 进行点乘，将得到的结果再作为一个特征值传给一个Sigmoid函数，经过此函数的转换，转换成一个值域在 $[0,1]$ 之间的一个值。这样就得到了 X_b 这组特征发生某一个特定事件的概率。

逻辑回归的表达式为： $\hat{p} = \sigma(X_b \cdot \theta)$ 。

这里的 σ 就是 Sigmoid 函数。接下来介绍为什么在二分类问题中选择 Sigmoid 函数作为预测分类函数。

Sigmoid 函数图像如下图所示：



Sigmoid 函数表达式为 $\phi(x) = \frac{1}{1+e^{-x}}$ 。由上图可知，Sigmoid 函数图像是一条取值在 0 和 1 之间的 S 形曲线。可以看出，Sigmoid 函数连续，光滑，严格单调，以 $(0,0.5)$ 为中心。当 x 趋近负无穷时， y 趋近于 0；当 x 趋近于正无穷时， y 趋近于 1。Sigmoid 函数的值域范围限制在 $(0,1)$ 之间，我们知道 $(0,1)$ 与概率值的范围是相对应的，这样 Sigmoid 函数就能与一个概率分布联系起来了。

2.2 损失函数

问题：对于给定的样本数据集 X, y ，如何找到参数 θ ，使得用这样的方式可以最大程度获得样本数据集 X 所对应的分类输出 y ？

在训练过程中的主要任务就是拟合训练样本，制定出这样的规则。在逻辑回归的框架下解决这个问题，远远没有像线性回归那么直观，因为对于线性回归来说可以非常容易的用真值减去预测值，用这个差的平方和来判断拟合的好坏，但是逻辑回归是一个分类问题，与线性回归的处理不一样。

$$\hat{p} = \sigma(X_b \cdot \theta) = \frac{1}{1 + e^{-X_b \cdot \theta}}$$
$$\hat{y} = \begin{cases} 1, & \hat{p} \geq 0.5 \\ 0, & \hat{p} \leq 0.5 \end{cases}$$

逻辑回归的大致框架是对于给定的样例 X_b ，求解出一组 θ ，让这个 X_b 点乘 θ 之后，再放到 Sigmoid 函数中，得到一个概率的估计值。如果概率的估计值 $\hat{p} \geq 0.5$ ，就将此样例 X_b 分类为 1；当 $\hat{p} < 0.5$ 时，将此样例 X_b 分类为 0。

在逻辑回归算法中有一个很重要的问题就是如何求出这个 θ 。在线性回归中 X_b 点乘 θ 之后

得到的就是预测值，让预测值减去真实值的差来度量估计的好坏，用这个差的平方和作为损失函数，求出让损失函数最小的 θ 即可。那么同理，对于逻辑回归也要这么做，整体的方向是一样的，只不过对于逻辑回归定义它的损失函数相对比较困难。逻辑回归和线性回归最大的区别就在于逻辑回归解决的是一个分类问题，估计出来的 $\hat{p}$ 是一个概率，来决定我们估计的这个 $\hat{y}$ 到底是 1 还是 0。它分成了两类，相应的损失函数也可以分为两类。

$$\text{损失函数} J(\theta) \begin{cases} \text{如果 } y=1, \hat{p} \text{ 越小, 损失函数 } J(\theta) \text{ 越大} \\ \text{如果 } y=0, \hat{p} \text{ 越大, 损失函数 } J(\theta) \text{ 越大} \end{cases}$$

如果 y 等于 1，表示样本的真实类别是 1，此时估计的 $\hat{p}$ 越小，即预测样本的类别趋近于 0，损失函数 $J(\theta)$ 越大。同理如果 y 等于 0，表示样本的真实类别是 0，此时估计的 $\hat{p}$ 越大，即预测样本的类别趋近于 1，估计错误，损失函数 $J(\theta)$ 越大。那么如何用函数来表达上述阐述的损失函数呢？

$$\text{损失函数} J(\theta) = \begin{cases} -\log(\hat{p}) & , y = 1 \\ -\log(1 - \hat{p}) & , y = 0 \end{cases}$$

为了看起来简洁，计算起来方便，还可以将上述的两个表达式整合成一个表达式。分别把 $y=1$ 和 $y=0$ 代入即可得到：

$$J(\theta) = -y \log(\hat{p}) - (1 - y) \log(1 - \hat{p})。$$

当真实值 $y=1$ 时，上述式子的第二项变成 0，此时损失函数变成 $J(\theta) = -\log(\hat{p})$ ；

当真实值 $y=0$ 时，上述式子的第一项变成 0，此时损失函数变成 $J(\theta) = -\log(1 - \hat{p})$ 。

将 $\hat{p} = \sigma(X_b \cdot \theta) = \frac{1}{1+e^{-X_b \cdot \theta}}$ 代入上式并用均方误差 MSE 来衡量损失函数可得：

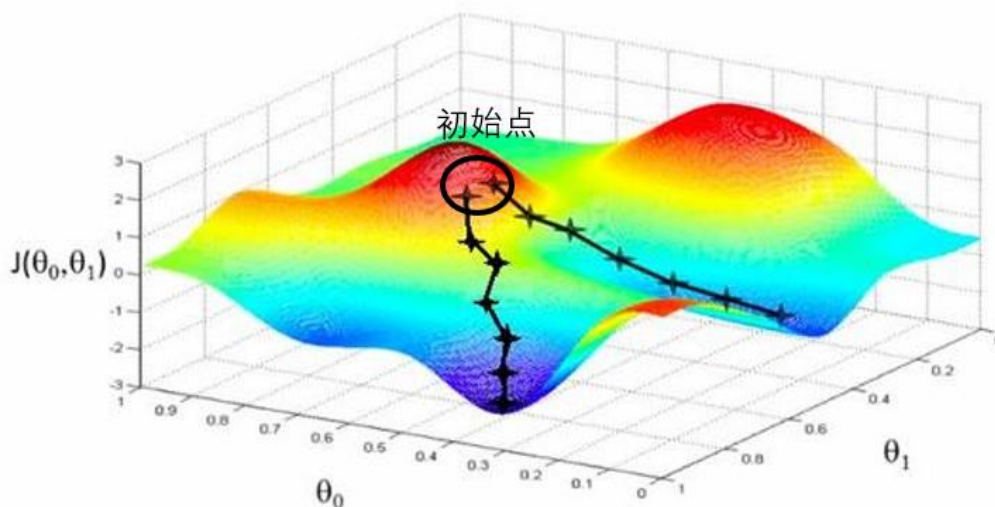
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \quad (1)$$

逻辑回归的损失函数不能像线性回归的损失函数一样非常容易地利用最小二乘法求出一个正规方程解，实际上对于这个式子来说，不能列出一个公式把 x 和 y 的值直接套进公式里得到 θ 。此时需要用到的方法是梯度下降法。

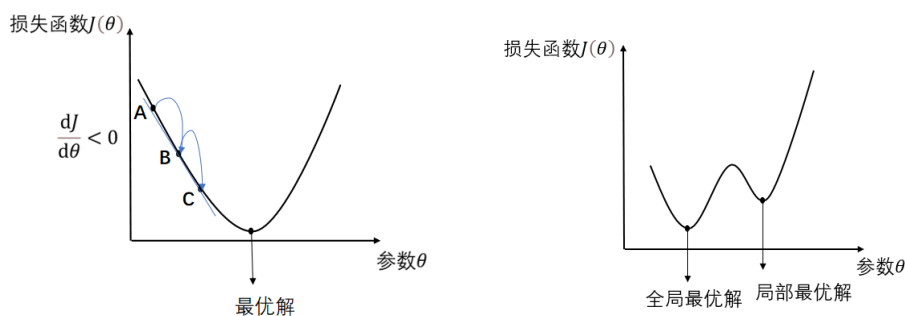
2.3 梯度下降法

上节已求出损失函数公式 $J(\theta)$ ，并且 $J(\theta)$ 是凸函数。如果损失函数 $J(\theta)$ 是凸函数，那么 $J(\theta)$ 一定存在函数最（极）小值。所以只要求出损失函数取最小值时的参数即完成任务。那函数的极值又怎么求呢？非常简单，当函数切线的斜率变为 0，也就是在损失函数的导数为 0 的地方，即为函数取得极值的地方（若自变量多于 1 个，导数也称为梯度）。在机器学习算法中，在最小化损失函数时，可以通过梯度下降法不断迭代求偏导求解，逐渐逼近 θ 的最佳值，以此得到最小化的损失函数和模型参数值。

下面来看一看梯度下降法的一个直观解释。比如在一座大山上的某处位置，由于不知道怎么下山，于是决定走一步看一步，也就是在每次走到一个位置的时候，求解当前位置的梯度，沿着梯度的负方向，就是当前最陡峭的位置向下走一步，然后继续求解当前位置梯度，向这一步所在位置沿着最陡峭最易下山的位置走一步。这样一步步的走下去，一直走到山脚的地方。当然这样走下去，有可能不能走到山脚，而是到了某一个局部的山峰低处。

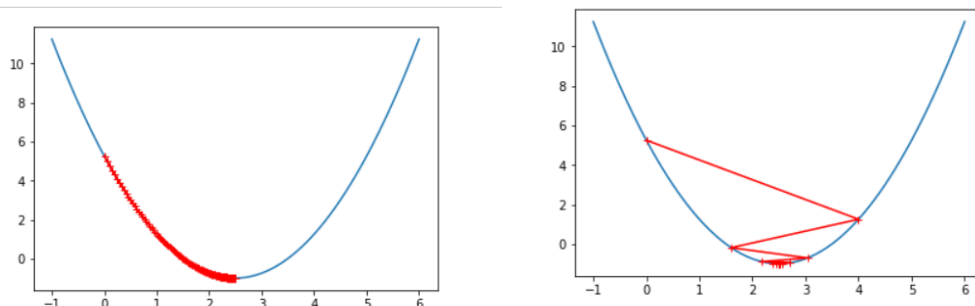


可以看出，梯度下降不一定能够找到全局的最优解，有可能是一个局部最优解。当然，如果损失函数是凸函数，梯度下降法得到的解就一定是全局最优解。如下图所示。



由上图知，并不是所有函数都有唯一的极值点，梯度下降法的初始点也是一个超参数。可通过多次运行，随机化初始点来解决这个问题。

这里再介绍两个其他的概念： η 表示学习率/步长， η 的取值影响获得最优解的速度。步长决定了要走多久才能到达目的地（损失函数取最小值处）。 η 是梯度下降法的一个超参数。如果步长 η 太小，则需要很长的时间才能到达目的地，如果 η 太大，可能导致在目的地的周围来回震荡。如下图所示。



$\frac{dJ}{d\theta}$ 表示方向/梯度，求梯度其实就是求损失函数中相应变量的偏导数。梯度决定是否走在正确的道路上，我们并没有上帝视角，因此只能找到当前的最佳梯度，但当前的梯度不一定是

回家的路，我们却只能前行。

$-\eta \cdot \frac{dJ}{d\theta}$ 来更新参数 θ ，不断地迭代更新以此求得损失函数的局部/全局最优解处。

表示的含义是利用梯度下降法来决定应该从哪个方向下山，每一步走多长。

为什么要沿着梯度的负方向移动？

假如从 A 点移动到 B 点，梯度 $\frac{dJ}{d\theta}$ 是小于 0 的， η 表示步长一定是正数，此时 $-\eta \cdot \frac{dJ}{d\theta}$ 是正数，表示 θ 应该往正数的方向移动，从 A 点到 B 点再到 C 点一步一步移动最终到达最优解，此时就是损失函数取极小值处。

链式求导法则： $f(g(x))' = f'(g(x)) \cdot g'(x)$ ，或者写成另外一种形式： $\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$

（如果有一个中间变量 z 可以沟通自变量 x 和因变量 y）。

$$\sigma(t) = \frac{1}{1+e^{-t}} = (1+e^{-t})^{-1} \quad \sigma(t)' = (1+e^{-t})^{-2} \cdot e^{-t}$$

$$\log(\sigma(t))' = \frac{1}{\sigma(t)} \cdot \sigma(t)' = 1 - \sigma(t) \quad \text{故} \quad \frac{\partial y^{(i)} \log(\sigma(X_b^{(i)} \theta))}{\partial \theta_j} = y^{(i)} (1 - \sigma(X_b^{(i)} \theta)) - x_j^{(i)}$$

针对某个 θ_j 进行求导， θ_j 前面有一个系数，是相应的 X_b 这个矩阵中第 i 行的第 j 列的数据，表示成 X_j^i 。

$$(\log(1 - \sigma(t)))' = \frac{1}{1 - \sigma(t)} \cdot (-1) \cdot \sigma'(t) = -\sigma(t) \quad (2)$$

$$\text{故} \quad \frac{\partial (1 - y^{(i)}) \log(1 - \sigma(X_b^{(i)} \theta))}{\partial \theta_j} = (1 - y^{(i)}) (-\sigma(X_b^{(i)} \theta)) x_j^{(i)} \quad (3)$$

$$\text{所以} \Delta J(\theta) = (1) \text{式} + (2) \text{式} = (y^{(i)} - \sigma(X_b^{(i)} \theta)) x_j^{(i)}$$

$$\text{扩展到所有样本的梯度为：} \frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) X_j^{(i)} \quad (4)$$

梯度的向量化表示：

$$\nabla J(\theta) = \frac{1}{m} \cdot \begin{pmatrix} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \\ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \cdot X_1^{(i)} \\ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \cdot X_2^{(i)} \\ \dots \\ \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) \cdot X_n^{(i)} \end{pmatrix} = \frac{1}{m} \cdot X_b^T \cdot (\sigma(X_b \theta) - y)$$

在梯度下降算法的处理过程中，可知梯度下降法在每次计算梯度时，都涉及全部样本。在样本数量特别大时，算法的效率会很低。**随机梯度下降法**（Stochastic Gradient Descent,SGD）试图改正这个问题，它不是通过计算全部样本来得到梯度，而是随机选择一个样本来计算梯度。随机梯度下降法不需要计算大量的数据，所以速度很快，但得到的并不是真正的梯度，可能会造成不收敛的问题。**批量梯度下降法**（Batch Gradient Descent,BGD）是一个折中的方法，每次在计算梯度时，选择小批量样本进行计算，既考虑了效率问题，又考虑了收敛问题。

2.4 决策边界

回顾逻辑回归分类的原理：通过训练的方式求出一个 $n+1$ 维向量 θ ，每当新来一个样本 X_b 时，与参数 θ 进行点乘，结果代入 Sigmoid 函数，得到的值为该样本发生 $\hat{y}$ 事件的概率值。如果概率大于 0.5，分类为 1；否则分类为 0。

对于公式： $\hat{p} = \frac{1}{1+e^{-t}}$ ($t=X_b \cdot \theta$)

当 $t>0$ 时, $1<1+e^{-t}<2$ ，即 $0.5<\hat{p}<1$ ；当 $t<0$ 时, $1+e^{-t}>2$ ，即 $0<\hat{p}<0.5$ 。

其中存在一个边界点 $X_b \cdot \theta=0$ ，当 $X_b \cdot \theta=0$ 时， $\hat{p}=0.5$ 。大于这个边界点分类为 1，小于这个边界点分类为 0。

边界点 $X_b \cdot \theta=0$ 称之为决策边界(Decision boundary)。

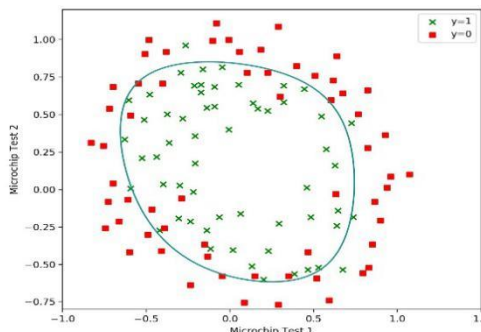
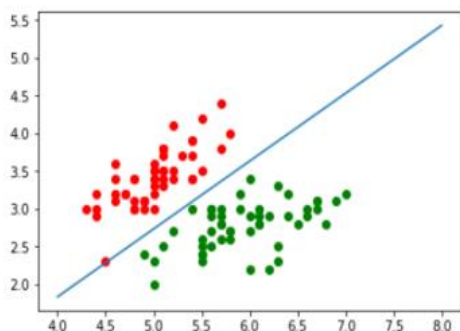
决策边界 $X_b \cdot \theta=0$ 同时也代表一个直线：假设 X 有两个特征 x_1, x_2 那么有 $X_b \cdot \theta = \theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$ 。这是一个直线，可以将数据集分成两类。可以改写成如下形式：

$$x_2 = \frac{-\theta_0 - \theta_1 x_1}{\theta_2}$$

所谓决策边界就是能够把样本进行正确分类的一条边界，主要有线性决策边界(Linear decision boundary)和非线性决策边界(Nonlinear decision boundary)。

一些算法，如逻辑回归或支持向量机 (Support Vector Machine, SVM) 可学习到线性决策边界；而另一些算法，如树型算法，可学习到非线性决策边界。**最好的决策边界是把新来的样本特征分到 0 的概率为 50%，分到 1 的概率也为 50%。**下面通过两幅图形象化的来说明线性决策边界和非线性决策边界。

逻辑回归的本质相当于在特征平面中，找到一条直线，用这条直线来分割所有的样本，只解决二分类问题，因为这条直线只能将平面分成两部分。如左下图所示。

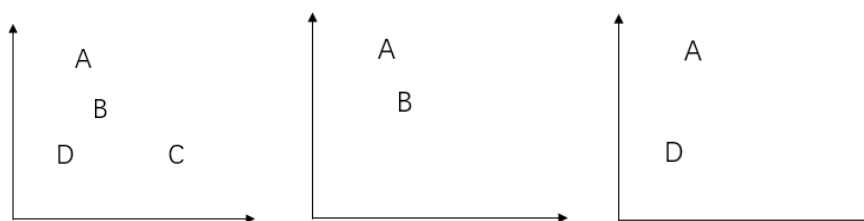


虽然线性决策边界容易理解，但仍要注意线性算法在非线性关系中会产生错误。存在的问题是直线的表达过于简单。可以引入多项式，将线性回归转换到多项式回归，利用这样的思想，应用到逻辑回归，可以对非线性数据进行一个比较好的分类，得到的决策边界也可能是曲线的形式。如右上图所示。

2.5 多分类逻辑回归

二分类问题是指预测值只有两个取值 (0 或 1)，二分类问题可以扩展到多分类问题。推广的方式有两种：分别叫做一对一 (One Vs One, OVO) 和一对其余 (One Vs Rest, OVR/One Vs All, OVA)。

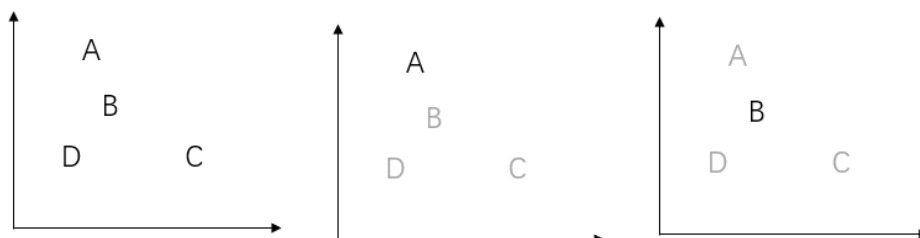
一对一 OVO 是将样本按照不同的类别进行拆分，并两两配对训练模型。



假如我们有 4 个不同的类别 A、B、C、D，每次只挑选 2 个类别进行二分类任务，对于这个二分类任务来说，我们判断新来的样本点属于哪一类的概率大，最终就划分为哪一类。如果

有 k 个类别，那么会生成 $C_k^2 = \frac{k(k-1)}{2}$ 个模型。

一对其余 OVR 是将其中一个类看成正类，若干个其他类都看成负类，训练得到一个二分类模型。



假如有 4 个不同的类别 A、B、C、D，那么可以生成 4 个不同的分类问题，每一次都选择其中的一个类别和剩余类别进行二分类。如上图所示，使用 OVR 解决多分类问题，首先取出 A 类将其与剩余所有类别进行二分类，对于每一个二分类任务，都能得到新来的样本点在每一个类别上的概率是多少，最后投票决定其类别。同样第二次取出 B 类将其与剩余所有类别进行二分类……由此可知： k 个类别就进行 k 次分类，最终选择分类得分最高的。

OVR 的计算复杂度提高了，如果处理一个二分类问题所用的时间为 t ，那么 k 个类别就需要使用 $k \cdot t$ 的时间来完成多分类任务。OVO 消耗的时间比 OVR 消耗的时间多，因为 $C_k^2 = \frac{k(k-1)}{2}$ 是一个 k^2 级别的运算，这使得最终的结果更加准确，因为每次都是用真实的类别进行比较，而没有混淆其他信息。

本章小结

逻辑回归是一个实现分类功能且应用非常广泛的机器学习算法，它将数据拟合到一个逻辑函数中，从而能够完成对事件发生的概率进行预测。逻辑回归是呈现离散变量之间相关关系的模型，二元结果为 0 或者 1 (扩大二元结果的概念可能得到多个结果)。因为名中带有“回归”，大家可能认为是回归的一种，但它是分类算法，它是线性回归算法的一种扩展。它始于输出结果为有实际意义的连续值的线性回归，但是线性回归对于分类的问题没有办法准确而又具备鲁棒性地分割，因此设计出了逻辑回归这样一个算法，它的输出结果表征了某个样本属于某类别的概率。例如，如果逻辑回归算法预测股价走势的结果值是 0.65，那么意味着股价走势“将上涨的概率是 65%”。

此时用到的 Sigmoid 函数就是预测分类函数，它的作用是将原本输出结果范围可以非常大的数据通过 Sigmoid 函数映射到 (0,1) 之间，从而完成对发生某事件概率的估测。求解逻辑回归参数的传统方法采用的是梯度下降法，构造为凸函数的损失函数后，每次沿着偏导负方向（下

降速度最快方向)迈进一小步,直至 n 次迭代后到达最低点。直观的在二维空间理解逻辑回归,是 Sigmoid 函数的特性,使得判定的阈值能够映射为平面的一条决策边界,当然随着特征的复杂化,决策边界可能是多种多样的形状,但是它能够较好地把两类样本点分隔开,解决分类问题。

下面总结了逻辑回归算法的优缺点。

优点：

- (1) 训练速度较快,分类的时候计算量仅仅只和特征的数目相关,计算代价不高。
- (2) 简单易理解,简单模型的可解释性非常好,易于实现。
- (3) 适合二分类问题,不需要缩放输入特征。
- (4) 内存资源占用小,因为只需要存储各个维度的特征值。

缺点：

- (1) 只能处理两分类问题,且必须线性可分,不能解决非线性问题;对于非线性特征,需要进行转换。
- (2) 很难处理数据不平衡的问题。
- (3) 当特征空间很大时,逻辑回归的性能不是很好,准确率并不是很高,因为形式非常的简单(非常类似线性模型),容易欠拟合。

习题

请结合自己的理解,简述一下什么是逻辑回归。

逻辑回归是一种用来解决二分类问题的机器学习方法,用于估计某事件发生的可能性。逻辑回归通过将样本的特征与样本发生的概率联系起来,输出样本可能发生的概率,根据输出概率的数值大小来推断新样本是属于哪一类别。如果输出的概率值大于 0.5,那么新输入的样本就属于这一类别,否则就不属于这一类别(逻辑回归用来判定类别的阈值不是只能设置 0.5,该阈值是一个超参数,可以自己调整)。